



Unsupervised Learning

CS 541 Lecture 3

Zining Zhu



Learning without labels

No labels · Structure discovery

What is unsupervised learning?

No labels, no “right answer” given at training time.

- Goal: find **structure** in the data itself
- Contrast with supervised learning: no target y to fit, only inputs
- Two big families we cover today
 - **Dimensionality reduction**: fewer axes, same signal
 - **Clustering**: group similar points together

Same data, different question: “what’s here?” instead of “what’s the label?”



Supervised vs. unsupervised

Aspect	Supervised	Unsupervised
Training data	(input, label) pairs	inputs only
Objective	match a known target	discover structure
Evaluation	held-out accuracy/loss	internal fit, or luck of having labels for validation
Typical outputs	a prediction	a projection, a partition, a density estimate
Failure mode	overfits the labels	“finds” structure that is not really there

Unsupervised learning has no scoreboard as clean as test accuracy, evaluation is one of the two hard problems in this lecture.



Why go looking for structure?

- **Compression:** store/transmit high-dim data more cheaply
- **Visualization:** humans can only see 2-3 dimensions at once
- **Anomaly detection:** outliers stand apart from discovered structure
- **Preprocessing:** feed cleaner, lower-dimensional features into a downstream model



Motivating applications: industry and consumer

- **Customer segmentation:** group users by behavior, no predefined segments
- **Visualization of embeddings:** word vectors, image features, projected to 2D
- **Recommendation priors:** cluster items before you have enough interactions to personalize
- **Document organization:** group articles/emails by topic without labeled categories



Motivating applications: science and safety

- **Anomaly detection:** fraud, network intrusion, defective manufactured parts
- **Genomics:** clustering gene expression profiles into putative cell types
- **Astronomy:** grouping stars/galaxies by observed properties before we know the physical category
- **Sensor monitoring:** flagging equipment behavior that departs from its usual pattern



A short history

- **1901:** Karl Pearson describes the idea behind PCA, fitting lines and planes by least squares to a cloud of points
- **1933:** Harold Hotelling names and formalizes **Principal Component Analysis**
- **1960s-70s:** cluster analysis formalized in the applied statistics literature (numerical taxonomy, pattern recognition)
- **1996:** DBSCAN introduces density-based clustering
- Today: the same core ideas, applied to million-dimensional embeddings

The math is a century old; what changed is the size and shape of the data we point it at.



Two families, previewed side by side

- Same six points, two different questions
 - **Dimensionality reduction** asks: **which 1 or 2 axes capture most of what varies across these points?**
 - **Clustering** asks: **which points belong together?**
- Often used together: reduce dimensions first, then cluster in the smaller space
- Today's plan: PCA in depth, then K-means, then hierarchical and density-based clustering, then how to evaluate any of it



Dimensionality reduction: PCA

Fewer axes, same signal

The curse of dimensionality

- High-dimensional data is **sparse**: points spread thin, distances between them become less meaningful
- More dimensions → more parameters → more data needed to fill the space
- Many real datasets have **redundant** or **correlated** features (e.g. height in inches and height in cm)
- Question: can we find a smaller set of axes that capture most of the signal?



Redundant and correlated features

- If two features move together, they carry mostly the same information
- Example: a house dataset with both “square feet” and “number of rooms”, correlated, not independent
- Redundancy inflates dimensionality without adding real information
- **Principal Component Analysis** finds new axes that repack correlated features into fewer, uncorrelated ones



Two views of the same idea

Principal Component Analysis answers this with one solution, two framings:

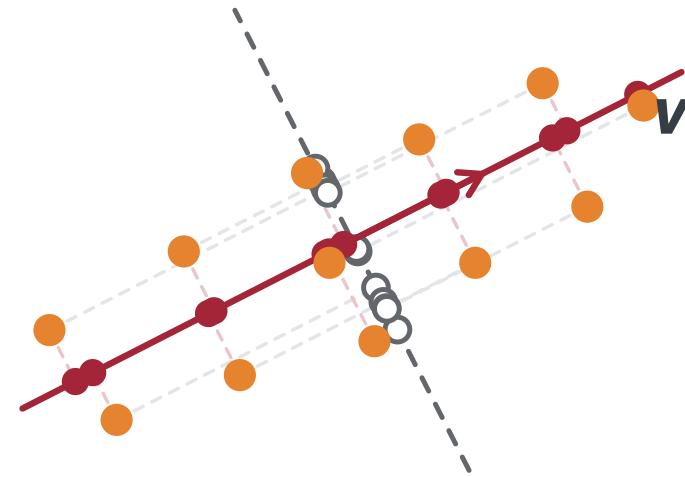
- **Maximize variance:** find directions where the data spreads out the most
- **Minimize reconstruction error:** find a low-dimensional subspace that loses the least information when we project back

Same optimal directions come out of either framing, this is not a coincidence, it falls out of the same optimization.



Variance framing

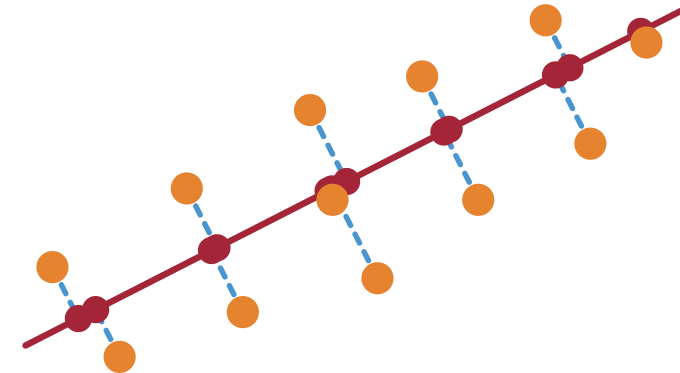
- Center the data (subtract the mean from every point)
- Find a unit direction v that maximizes the variance of the data once projected onto v
- Second component: same question, but restricted to be orthogonal to the first
- Repeat for as many components as needed, each orthogonal to all previous ones



wide spread on v , narrow on the other

Reconstruction framing

- Project the data down to k dimensions, then reconstruct back to the original space
- Minimize the squared error between the original points and their reconstructions
- The same directions v pop out as in the variance framing
- Intuition: we are throwing away the **least** informative directions first



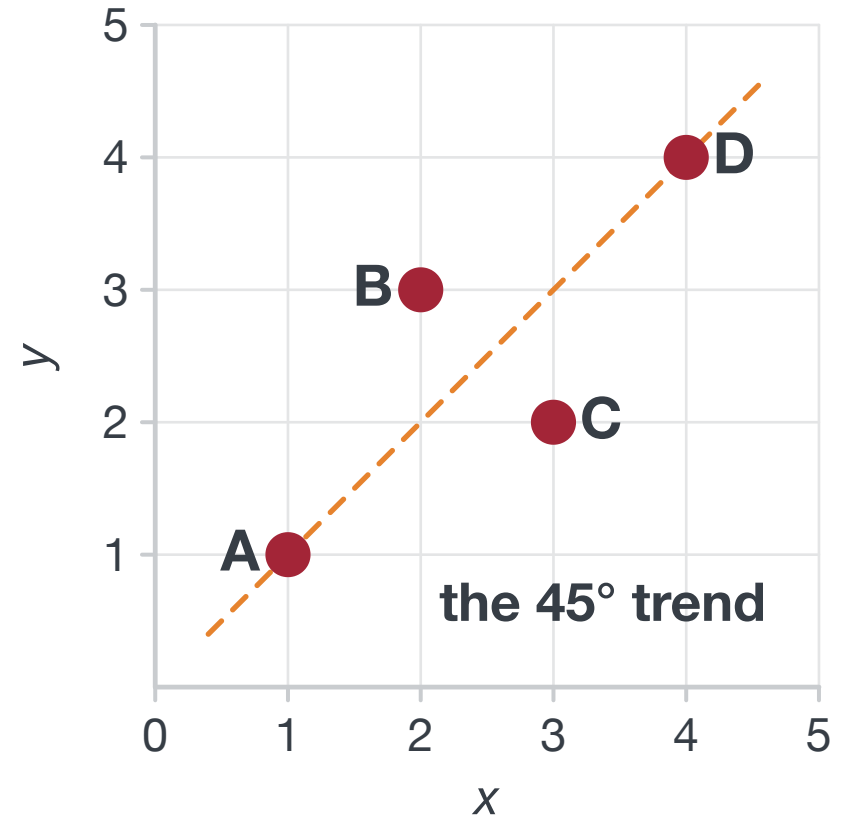
- the point
 - its reconstruction
- blue drops are what the projection throws away

A worked example: the data

Four points in two dimensions:

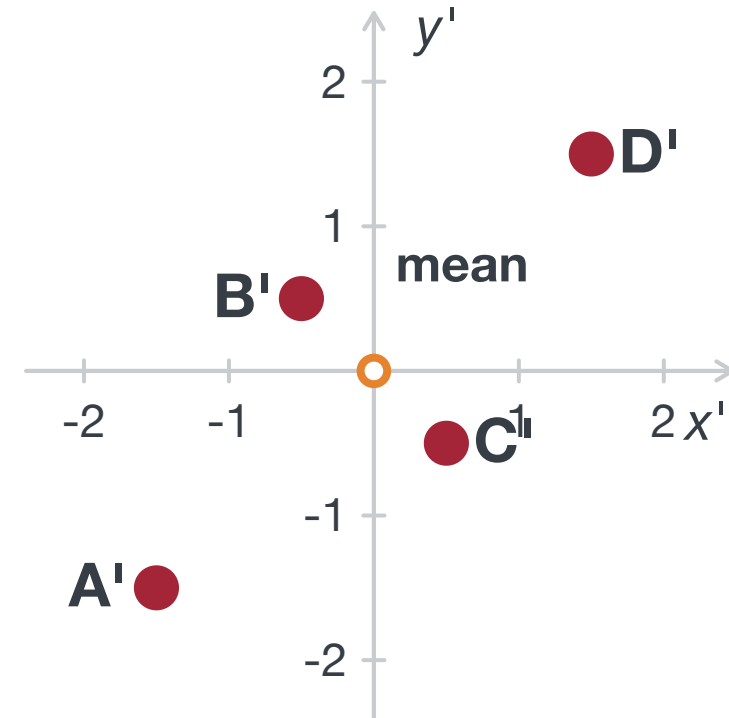
Point	x	y
A	1	1
B	2	3
C	3	2
D	4	4

Notice the points roughly track the 45° diagonal, that visual intuition will reappear in the eigenvectors.



Worked example: mean and centering

- Mean: $\text{mean}(x) = (1+2+3+4)/4 = 2.5$,
 $\text{mean}(y) = (1+3+2+4)/4 = 2.5$
- Center every point by subtracting the mean:
 - $A' = (-1.5, -1.5)$
 - $B' = (-0.5, 0.5)$
 - $C' = (0.5, -0.5)$
 - $D' = (1.5, 1.5)$
- Centering is the first step of **every** PCA computation, never skip it

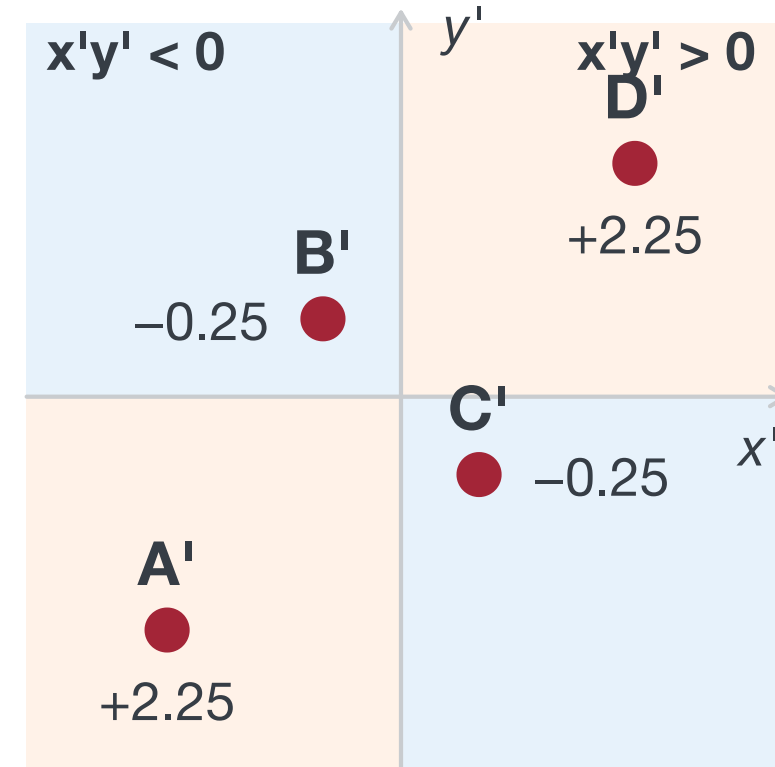


Worked example: the covariance matrix

Using the population convention (divide by $n = 4$):

- $\text{Var}(x) = \text{mean of } x'^2 = (2.25 + 0.25 + 0.25 + 2.25) / 4 = 1.25$
- $\text{Var}(y) = \text{mean of } y'^2 = (2.25 + 0.25 + 0.25 + 2.25) / 4 = 1.25$
- $\text{Cov}(x, y) = \text{mean of } x'y' = (2.25 - 0.25 - 0.25 + 2.25) / 4 = 1.00$

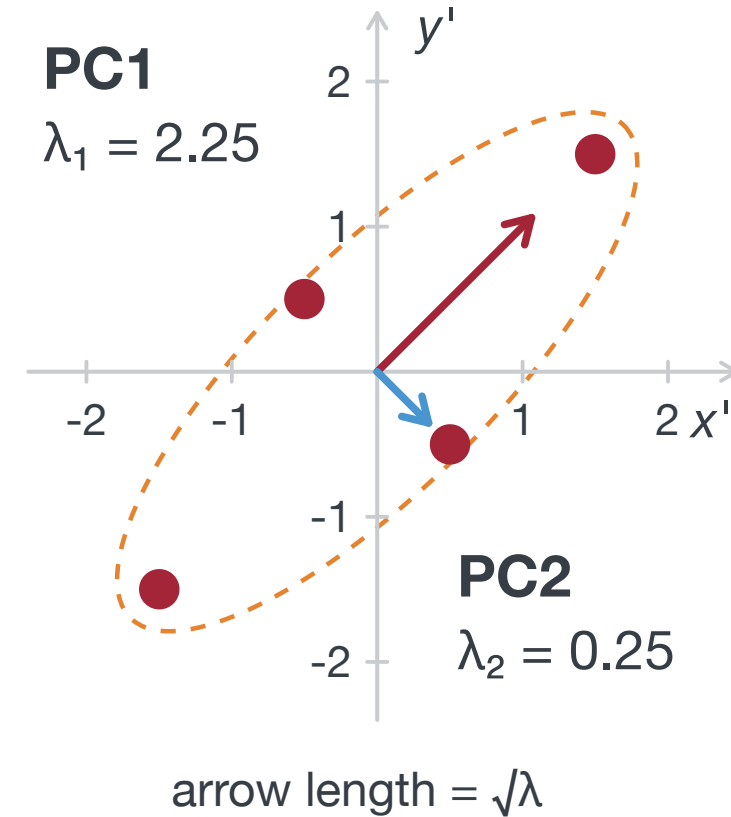
The covariance matrix is $[[1.25, 1.00], [1.00, 1.25]]$, symmetric, as covariance matrices always are.



Worked example: eigenvectors and eigenvalues

For a symmetric 2×2 matrix with equal diagonal entries a and off-diagonal b , the eigenvalues are $a+b$ and $a-b$, with eigenvectors along the diagonal and anti-diagonal:

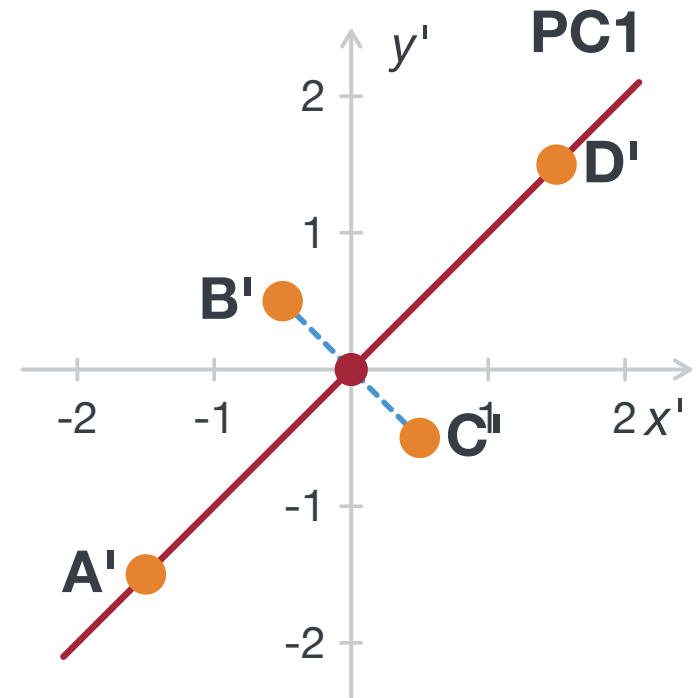
- $\lambda_1 = 1.25 + 1.00 = \mathbf{2.25}$, eigenvector direction $(1, 1)$, the 45° diagonal
- $\lambda_2 = 1.25 - 1.00 = \mathbf{0.25}$, eigenvector direction $(1, -1)$, the anti-diagonal
- Total variance = $\lambda_1 + \lambda_2 = 2.50$, matching the trace of the covariance matrix ($1.25 + 1.25$)



Worked example: principal axes and explained variance

- **PC1** points along (1, 1): the direction the data actually spreads out along, matching the visual diagonal trend
- **PC2** points along (1, -1): the leftover spread once PC1 is accounted for
- Explained variance ratio: $PC1 = 2.25 / 2.50 = \mathbf{90\%}$, $PC2 = 0.25 / 2.50 = \mathbf{10\%}$
- Keeping only PC1 would compress these 4 points from 2 numbers each down to 1, losing only 10% of the variance

This is the entire PCA recipe, done by hand, on four points. Real datasets just repeat this at larger scale.



PC1 90%

PC2 10%

Eigenvectors and eigenvalues, in general

- Compute the covariance matrix of the (centered) data, of size (number of features) $\times$ (number of features)
- Its **eigenvectors** are the principal component directions
- Its **eigenvalues** measure how much variance each direction explains
- Sort eigenvalues in decreasing order $\rightarrow$ that ordering **is** the importance ranking of the components

We will not derive eigendecomposition from scratch; the intuition (variance-maximizing directions) matters more than the linear algebra proof here.



Covariance matrix vs. correlation matrix

- The covariance matrix is sensitive to the **units** each feature is measured in
- The correlation matrix is the covariance matrix computed on standardized (zero mean, unit variance) features
- Rule of thumb: if features are on different scales or units, run PCA on the correlation matrix (i.e. standardize first)
- If all features already share comparable units and scales, the covariance matrix is fine as is



Explained variance ratio: a bigger worked example

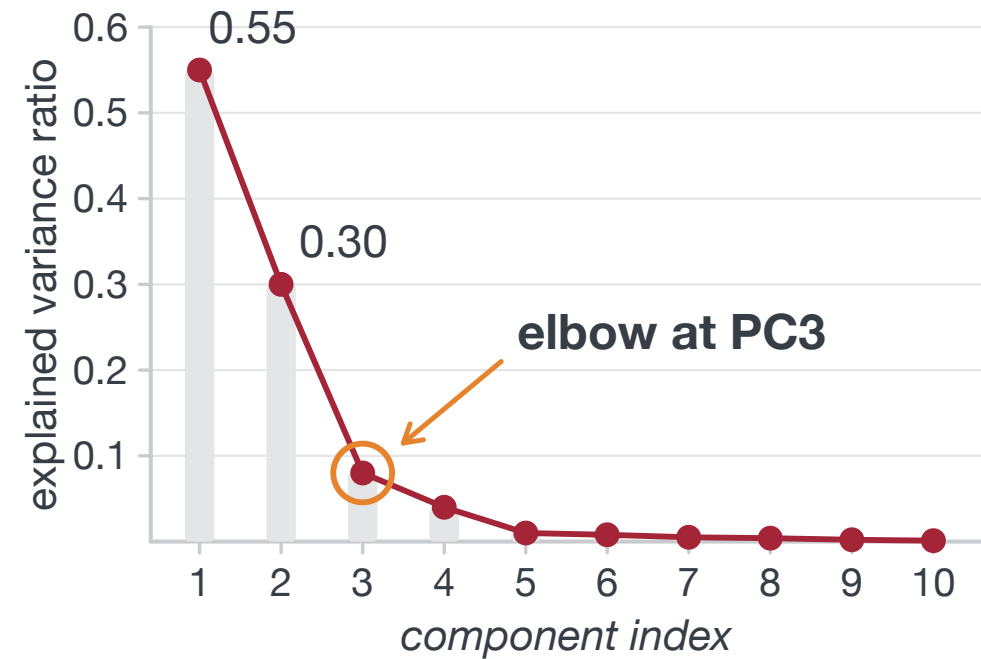
Ten original features, PCA run, eigenvalues sorted and normalized to sum to 1:

Component	Eigenvalue share
PC1	0.55
PC2	0.30
PC3	0.08
PC4	0.04
PC5-PC10 (combined)	0.03

- Cumulative variance after 2 components: $0.55 + 0.30 = 85\%$
- Cumulative variance after 3 components: **93%**
- A common rule of thumb: keep enough components to cross a chosen threshold, often 90-95%

Choosing how many components

- Plot the **explained variance ratio** per component in decreasing order (a **scree plot**)
- Look for an “elbow” where the curve flattens, or pick a cumulative variance threshold (e.g. 90%)
- Trade-off: fewer components = more compression, more information loss



Why standardize first: a worked mini example

- Feature 1: house price in dollars, typical range 200,000-800,000 (variance in the billions)
- Feature 2: number of bedrooms, typical range 1-5 (variance under 2)
- Without standardizing, PC1 will point almost entirely along “price”, its huge variance dominates, regardless of how informative “bedrooms” actually is
- After standardizing both features to unit variance, PCA compares them on equal footing

Scale-sensitivity is not a corner case, it is the default failure mode of PCA on raw, unscaled data.



Limitations of PCA

- **Linear only:** cannot capture curved or nonlinear structure (e.g. a spiral)
- **Scale-sensitive:** features on bigger scales dominate variance unless standardized first
- Components are linear combinations of **all** original features → often harder to interpret than the raw features
- Assumes the interesting structure is where the variance is largest, not always true (e.g. class-discriminative structure can hide in a low-variance direction)



PCA in high dimensions

- If the number of features p is much larger than the number of samples n (common in genomics, text embeddings), the covariance matrix is rank-deficient
- At most $n-1$ meaningful principal components exist, regardless of how large p is
- Practical fix: compute PCA via the singular value decomposition of the data matrix directly, which handles this case efficiently



PCA as a recipe

1. Center the data (and standardize, if features are on different scales)
2. Compute the covariance (or correlation) matrix
3. Compute its eigenvectors and eigenvalues
4. Sort eigenvectors by descending eigenvalue
5. Choose k components via a scree plot or a cumulative variance threshold
6. Project the data onto the top k eigenvectors



Beyond PCA: nonlinear alternatives

- **t-SNE**: preserves local neighborhoods for visualization, not a general-purpose feature reducer
- **UMAP**: similar goal to t-SNE, often faster and better at preserving some global structure
- **Autoencoders**: a neural network learns a nonlinear compression, trained by reconstruction error
- Mentioned today for context; not covered in depth, PCA remains the right first tool to reach for



Check your understanding: PCA

- Why must the data be centered before computing the covariance matrix?
- In the worked example, why did PC1 explain 90% of the variance rather than 50%?
- Give one situation where you would standardize before PCA, and one where you would not



Clustering: K-means

K-means · Assign and update

The clustering problem

- Partition n points into K groups
- Points in the same group should be **similar**; points in different groups, **dissimilar**
- No labels tell us what the groups should be, we define “similar” ourselves, usually via distance



Measuring similarity: distance metrics

- **Euclidean distance:** straight-line distance, the default for continuous features
- **Manhattan distance:** sum of absolute coordinate differences, more robust to outliers along any one axis
- **Cosine distance:** measures the angle between vectors, ignoring magnitude, common for text/embedding data
- The choice of distance metric is itself a modeling decision, not a neutral default



The K-means objective

- Pick K cluster centers (**centroids**)
- Assign each point to its nearest centroid
- Objective, called **inertia**: minimize the total squared distance from every point to its assigned centroid
- This is a combinatorial optimization problem over all possible assignments, an exact solution is too expensive to compute directly



Lloyd's algorithm

- **Initialize:** pick K centroids (e.g. randomly, among the data points)
- **Assign step:** assign each point to its nearest centroid
- **Update step:** move each centroid to the mean of the points currently assigned to it
- Repeat the assign/update loop until centroids stop moving (or moves fall below a tolerance)

Each step never increases the objective, the algorithm is guaranteed to converge, but only to a **local** optimum.

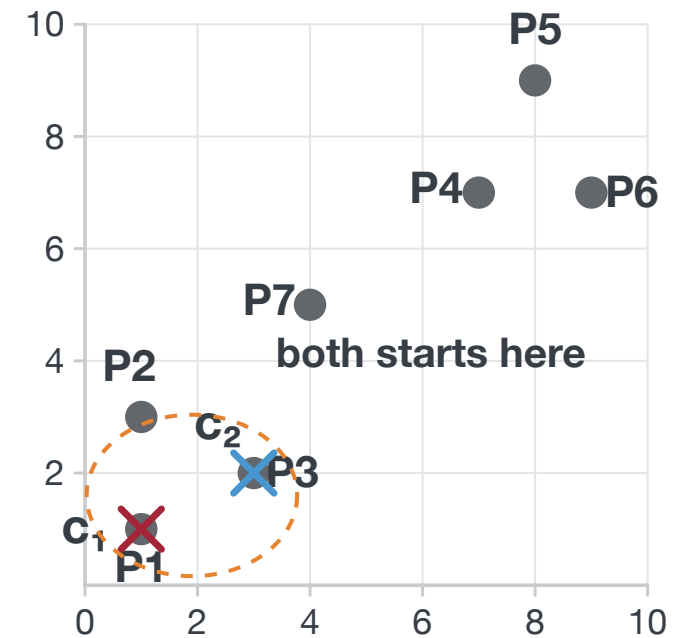


Worked example: the data and a poor initialization

Seven points, and two initial centroids chosen directly from the data:

Point	x	y
P1	1	1
P2	1	3
P3	3	2
P4	7	7
P5	8	9
P6	9	7
P7	4	5

Initial centroids: $c1 = P1 = (1, 1)$, $c2 = P3 = (3, 2)$, both accidentally near the same corner.

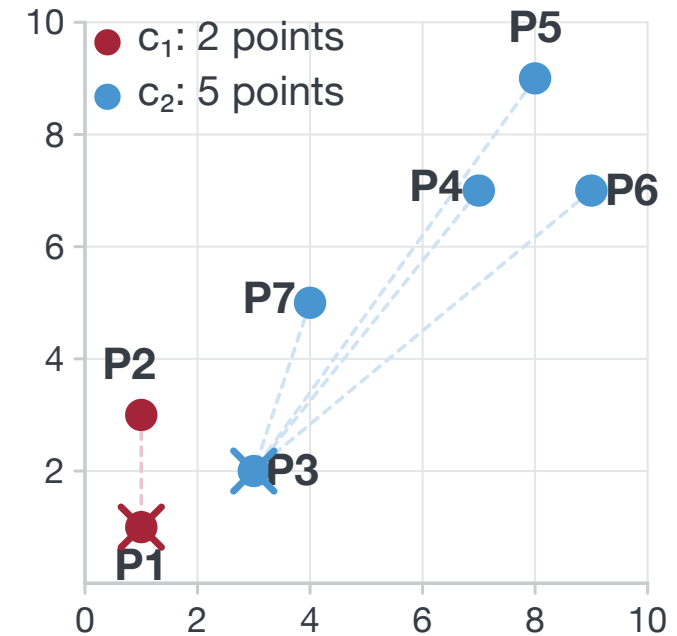


Worked example: iteration 1, assign step

Distance from each point to $c_1 = (1,1)$ and $c_2 = (3,2)$:

Point	to c_1	to c_2	assigned
P1	0.00	2.24	c_1
P2	2.00	2.24	c_1
P3	2.24	0.00	c_2
P4	8.49	6.40	c_2
P5	10.63	8.60	c_2
P6	10.00	7.81	c_2
P7	5.00	3.16	c_2

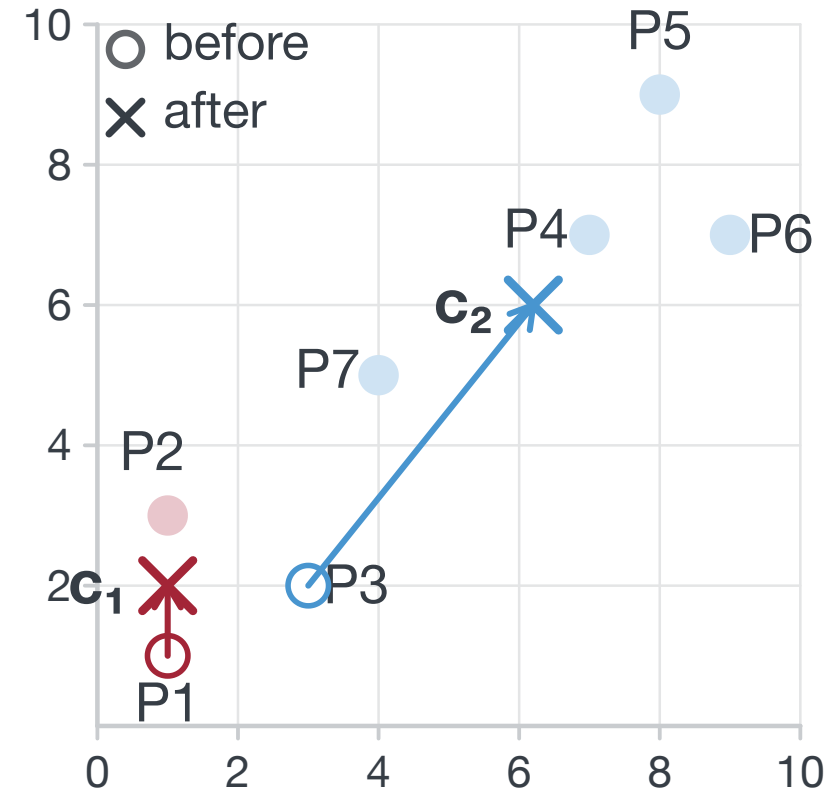
The poor initialization leaves cluster 2 with five points and cluster 1 with only two, the update step will fix this.



Worked example: iteration 1, update step

- New $c_1 = \text{mean}(P_1, P_2) = ((1+1)/2, (1+3)/2) = \mathbf{(1.00, 2.00)}$
- New $c_2 = \text{mean}(P_3, P_4, P_5, P_6, P_7) = ((3+7+8+9+4)/5, (2+7+9+7+5)/5) = \mathbf{(6.20, 6.00)}$

Both centroids moved substantially, the algorithm is still far from converged after one pass.

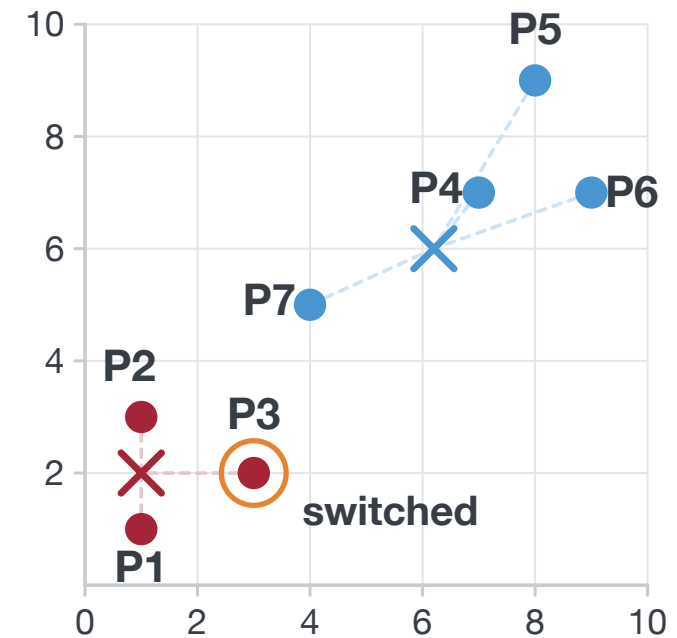


Worked example: iteration 2, assign step

Distance from each point to the **updated** centroids
 $c1 = (1,2)$, $c2 = (6.2,6.0)$:

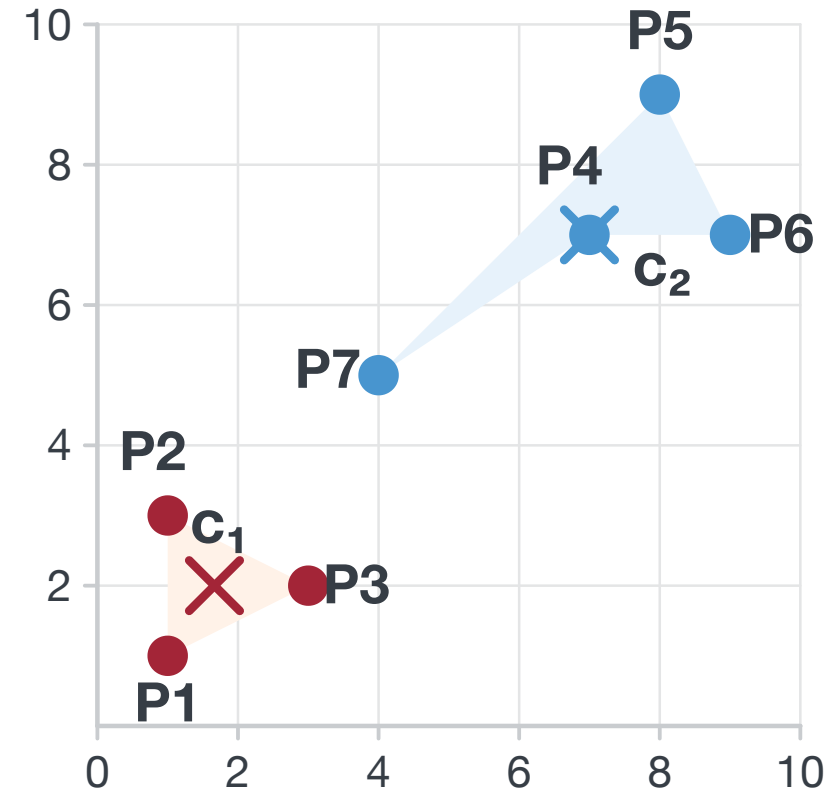
Point	to c1	to c2	assigned
P1	1.00	7.21	c1
P2	1.00	6.00	c1
P3	2.00	5.12	c1 (switched!)
P4	7.81	1.28	c2
P5	9.90	3.50	c2
P6	9.43	2.97	c2
P7	4.24	2.42	c2

P3 switches from c2 to c1, this is exactly why we iterate rather than stopping after one pass.



Worked example: iteration 2, update, and convergence

- New $c_1 = \text{mean}(P_1, P_2, P_3) = (1.67, 2.00)$
- New $c_2 = \text{mean}(P_4, P_5, P_6, P_7) = (7.00, 7.00)$
- Checking iteration 3's assign step against these centroids reassigns no points, the algorithm has **converged**
- Final clusters: $\{P_1, P_2, P_3\}$ (bottom-left) and $\{P_4, P_5, P_6, P_7\}$ (top-right)



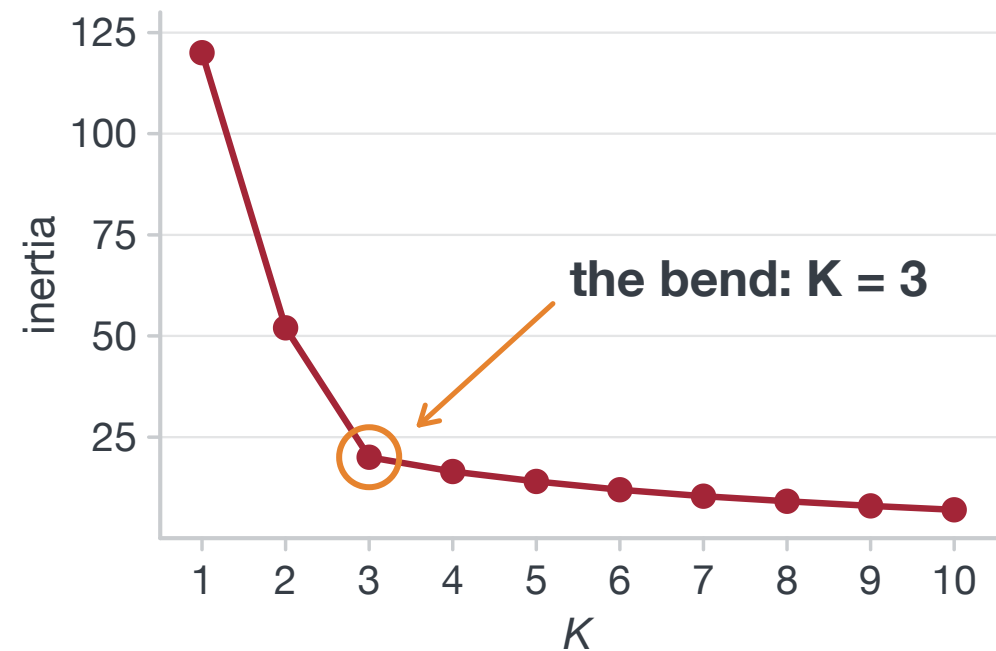
Why Lloyd's algorithm converges

- The assign step can only decrease (or hold) total squared distance, since every point moves to its **nearest** centroid
- The update step can only decrease (or hold) total squared distance, since the mean is the point that minimizes squared distance to a fixed set of points
- Inertia is bounded below by zero and strictly decreases (or the algorithm stops), so it must converge in finitely many steps
- What it does **not** guarantee: convergence to the **global** minimum



Choosing K: the elbow method

- Run K-means for a range of K values (e.g. 1 through 10)
- Plot inertia against K
- Inertia always decreases as K grows (more centroids can only fit the data at least as well), look for the point where the curve stops dropping quickly
- That bend is the “elbow”, a rough, not exact, guide



Choosing K: the silhouette score

For a point i , let $a(i)$ be its average distance to points in its **own** cluster, and $b(i)$ its average distance to points in the **nearest other** cluster:

- $\text{silhouette}(i) = (b(i) - a(i)) / \max(a(i), b(i))$
- Ranges from -1 to 1: near 1 means well-clustered, near 0 means on a cluster boundary, negative means likely misassigned
- Average silhouette score across all points, compare across candidate K values, pick the K with the highest average



Silhouette score: a worked mini example

Take point $P1 = (1,1)$ from the earlier example, in the final clustering $\{P1,P2,P3\}$ vs. $\{P4,P5,P6,P7\}$:

- $a(P1) = \text{average distance to } P2, P3 = (2.00 + 2.24) / 2 = 2.12$
- $b(P1) = \text{average distance to } P4, P5, P6, P7 = (8.49 + 10.63 + 10.00 + 5.00) / 4 = 8.53$
- $\text{silhouette}(P1) = (8.53 - 2.12) / \max(2.12, 8.53) = 6.41 / 8.53 \approx \mathbf{0.75}$

A silhouette near 0.75 says $P1$ sits much closer to its own cluster than to the other one, a good sign for this clustering.



Limitations of K-means

- Assumes clusters are roughly **spherical** and similarly sized
- Sensitive to **initialization**, a bad starting point can lead to a bad local optimum
- Must choose K in advance
- Sensitive to feature scaling, for the same reason PCA is



Fixing initialization: k-means++

1. Pick the first centroid uniformly at random from the data points
2. For each remaining point, compute its squared distance to the nearest centroid chosen so far
3. Pick the next centroid at random, with probability proportional to that squared distance
4. Repeat steps 2-3 until K centroids are chosen, then run Lloyd's algorithm as usual

Points far from existing centroids are more likely to be picked next, this spreads the initial centroids apart on purpose.



k-means++ worked mini example

Using the seven points from before, suppose the first centroid picked at random is $P1 = (1,1)$:

- Squared distances from $P1$: $P2 = 4$, $P3 = 5$, $P4 = 72$, $P5 = 113$, $P6 = 100$, $P7 = 25$
- $P5$ has by far the largest squared distance (113), so it has the highest chance of being picked next
- If $P5$ is picked, the two initial centroids ($P1$ and $P5$) already sit in the two different natural clusters, no bad initialization this time



K-medoids: a robustness alternative

- K-means centroids are means, which are pulled by outliers
- **K-medoids** restricts each cluster center to be an actual data point (a **medoid**), chosen to minimize total distance to its cluster's points
- More robust to outliers, works with any distance metric (not just Euclidean), but more expensive per iteration
- A good substitute when the data has heavy outliers or a non-Euclidean distance makes more sense



Scaling up: Mini-batch K-means

- Standard Lloyd's algorithm scans **every** point at every iteration, expensive for millions of points
- **Mini-batch K-means** updates centroids using small random batches of points at each step, similar in spirit to stochastic gradient descent from lecture 2
- Trades a small amount of clustering quality for a large speedup on big datasets
- Same objective, same guarantees in spirit, different computational budget



Check your understanding: K-means

- Walk through why P3 switched clusters between iteration 1 and iteration 2 in the worked example
- Why does inertia alone not tell you the “right” K?
- What problem does k-means++ solve, and what problem does it **not** solve?



Other clustering approaches

Hierarchies · Density

Hierarchical clustering: the idea

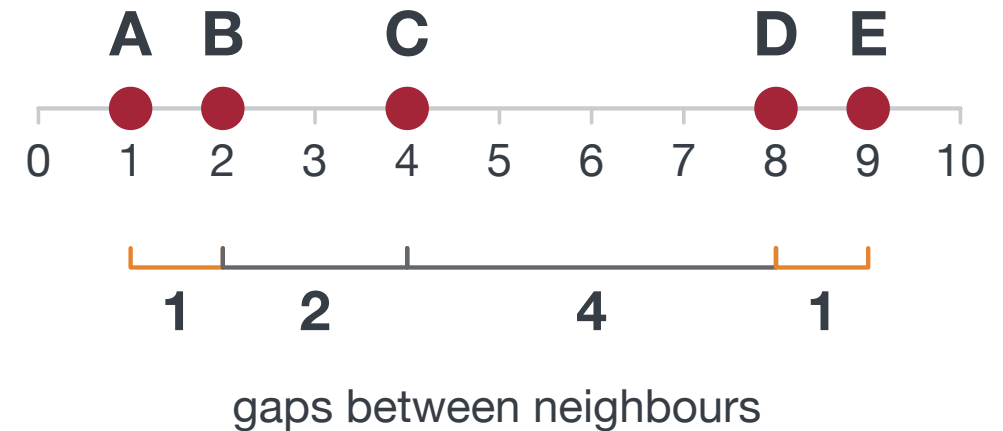
- Build a tree of clusters instead of committing to one fixed partition
- **Agglomerative** (bottom-up): start with every point as its own cluster, repeatedly merge the closest pair
- Result: a **dendrogram**, cut it at any height to get a clustering with however many clusters that height implies
- No need to choose K up front; choose the cut height instead



Worked example: five points and their distances

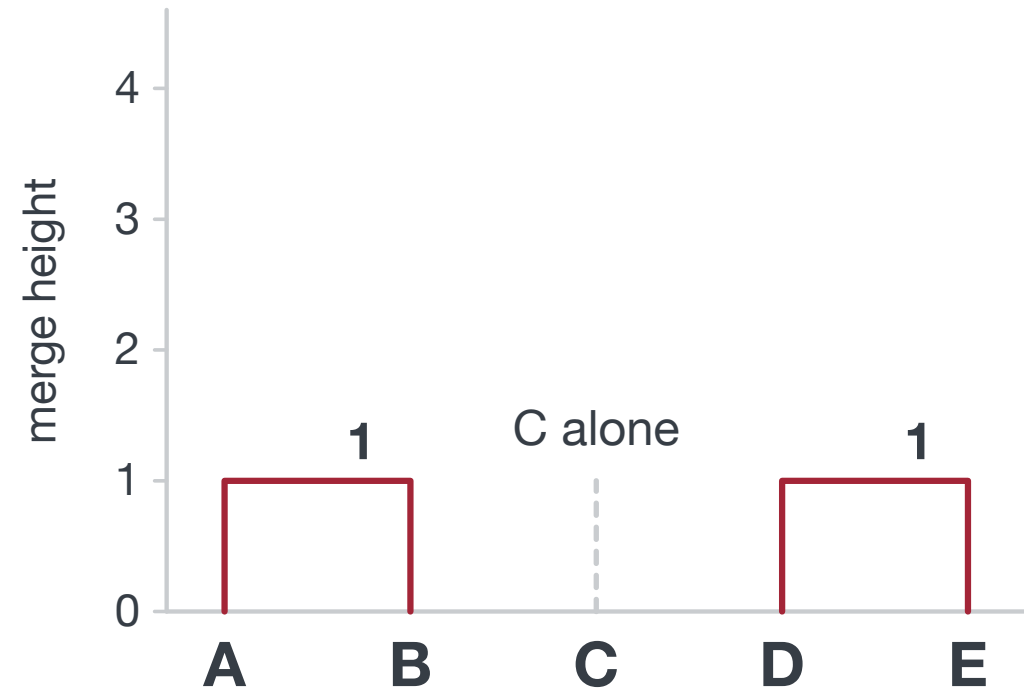
Five points on a number line: A=1, B=2, C=4, D=8, E=9. Pairwise distances (absolute difference):

	A	B	C	D	E
A	-	1	3	7	8
B	1	-	2	6	7
C	3	2	-	4	5
D	7	6	4	-	1
E	8	7	5	1	-



Worked example: the first merges

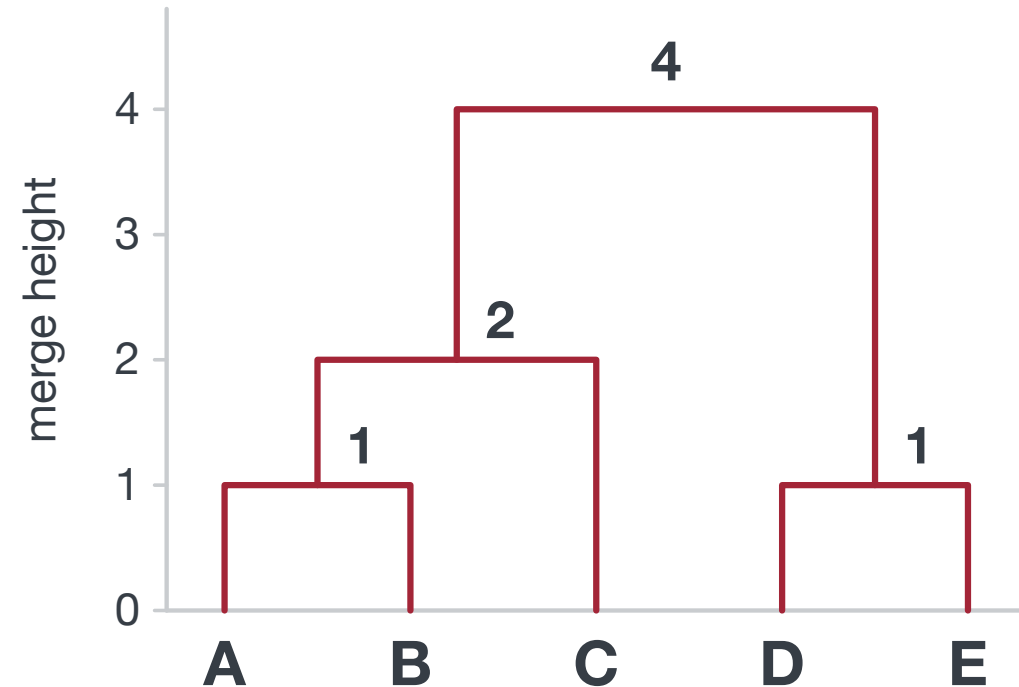
- Smallest distance in the table is 1, tied between (A,B) and (D,E)
- Merge A and B into cluster {AB} at height 1
- Merge D and E into cluster {DE} at height 1
- Remaining clusters: {AB}, {C}, {DE}



Worked example: single linkage, completing the tree

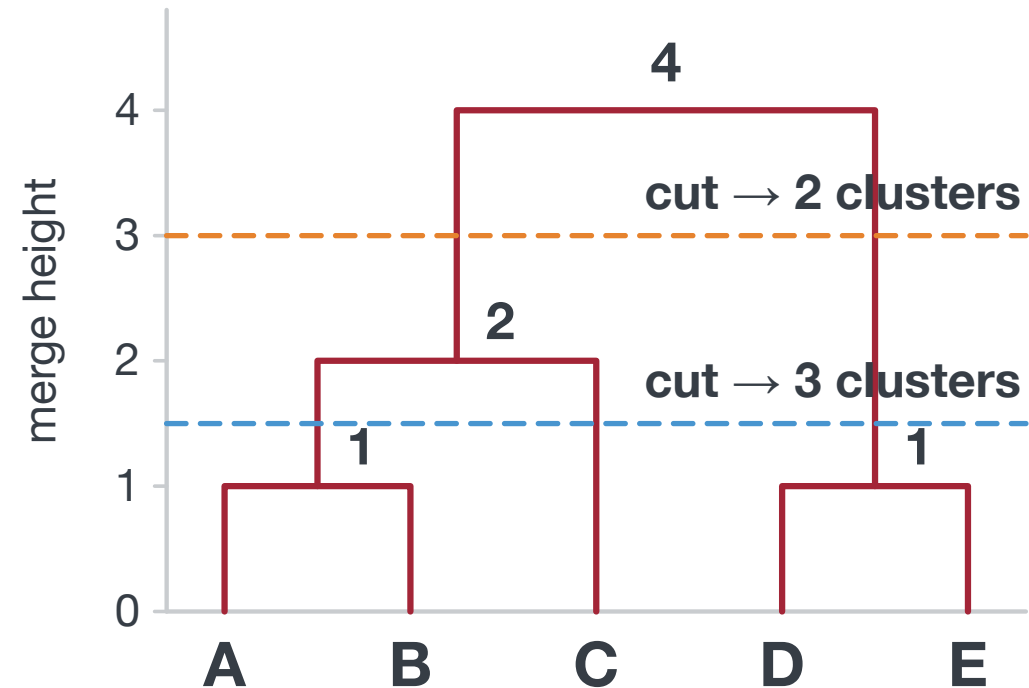
Single linkage distance = minimum distance between any pair across two clusters:

- $d(\{AB\}, C) = \min(AC, BC) = \min(3, 2) = 2$
- $d(\{AB\}, \{DE\}) = \min(AD, AE, BD, BE) = \min(7, 8, 6, 7) = 6$
- $d(C, \{DE\}) = \min(CD, CE) = \min(4, 5) = 4$
- Next merge: $\{AB\}$ with C at height 2, giving $\{ABC\}$
- Final merge: $\{ABC\}$ with $\{DE\}$ at height 4, giving one cluster



Cutting the dendrogram

- A cut between heights 2 and 4 yields two clusters: {A, B, C} and {D, E}
- A cut just above height 1 yields three clusters: {A, B}, {C}, {D, E}
- The dendrogram encodes **every** possible number of clusters at once, the cut height is the only choice left to make



Linkage criteria compared

How do we measure “distance between clusters” when deciding what to merge next?

- **Single linkage:** distance between the closest pair of points, tends to produce elongated, chained clusters, sensitive to noise
- **Complete linkage:** distance between the farthest pair of points, tends to produce compact, evenly sized clusters
- **Average linkage:** average pairwise distance between all cross-cluster pairs, a middle ground between the two

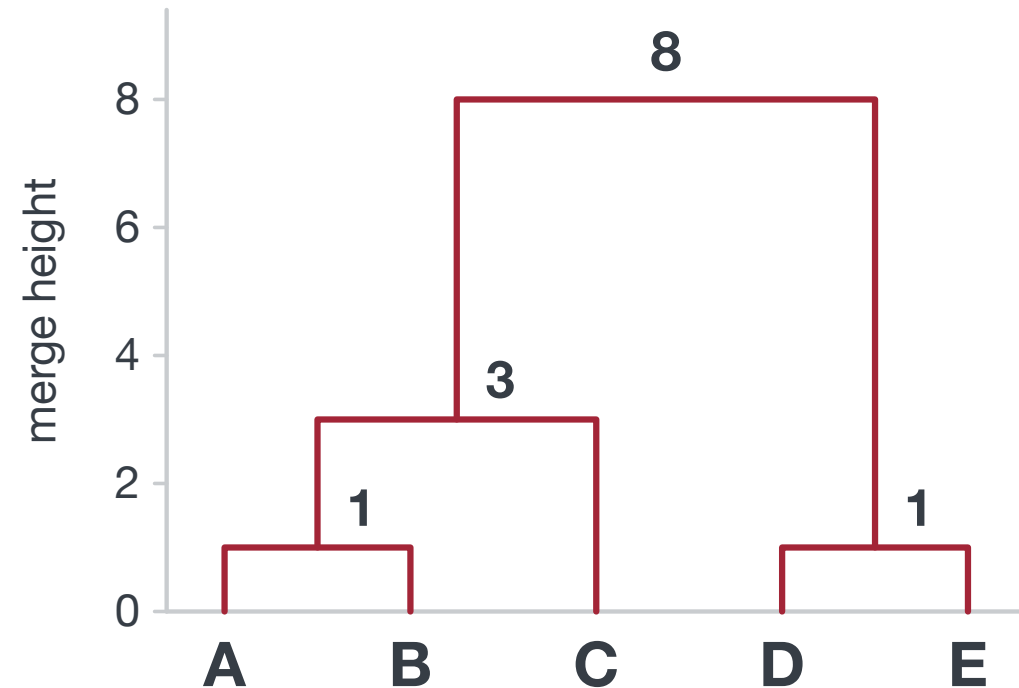


Same data, different linkage: a worked contrast

Rerunning the five-point example with **complete linkage** instead of single linkage:

- $d(\{AB\}, C) = \max(AC, BC) = \max(3, 2) = 3$
- $d(\{AB\}, \{DE\}) = \max(AD, AE, BD, BE) = \max(7, 8, 6, 7) = 8$
- $d(C, \{DE\}) = \max(CD, CE) = \max(4, 5) = 5$
- Merges: $\{AB\}$ with C at height **3** (not 2), then final merge at height **8** (not 4)

Same data, same first two merges, but the tree's heights, and the natural cut points, shift. Linkage choice changes the answer.



Divisive clustering: the mirror image

- Agglomerative clustering works bottom-up: start with singletons, merge upward
- **Divisive** clustering works top-down: start with all points in one cluster, repeatedly split the least cohesive one
- Divisive methods are less common in practice (each split is itself a smaller clustering problem to solve), but conceptually complete the picture



DBSCAN: density-based clustering

- Clusters are regions of high point density, separated by regions of low density
- Two parameters: neighborhood radius ϵ (epsilon), and **minPts**, the minimum number of points (including itself) required within that radius for a point to be considered dense
- No need to specify the number of clusters in advance



Core, border, and noise points

- A **core point** has at least minPts points (including itself) within distance ϵ
- A **border point** is not itself core, but falls within ϵ of some core point, it joins that core point's cluster
- A **noise point** is neither core nor within ϵ of any core point, it is not assigned to any cluster

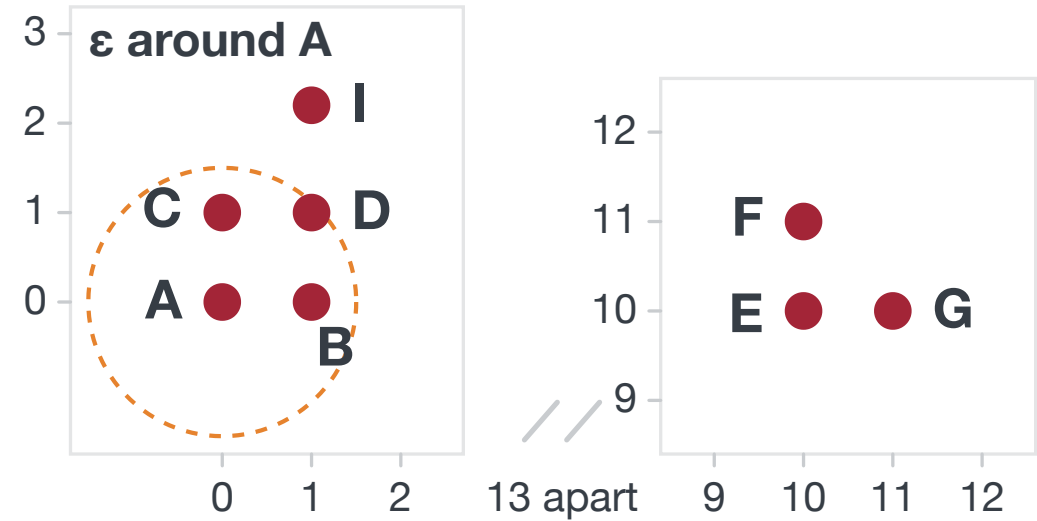


DBSCAN worked example: the setup

Eight points, $\epsilon = 1.5$, minPts = 3:

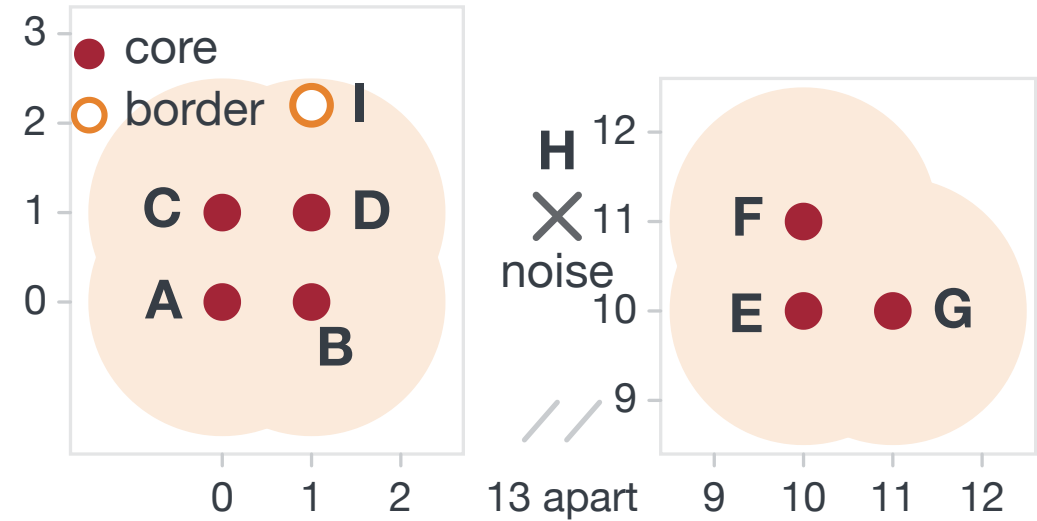
Point	x	y
A	0	0
B	1	0
C	0	1
D	1	1
I	1	2.2
E	10	10
F	10	11
G	11	10

H = (5,5), an isolated point far from everything, is added on the next slide.



DBSCAN worked example: classifying the points

- **A, B, C, D**: every pairwise distance among them is at most $\sqrt{2} \approx 1.41 < \epsilon$, each has 3 neighbors within ϵ (including itself) $\rightarrow$ all four are **core points**, one cluster
- **E, F, G**: same story, mutually within $\epsilon \rightarrow$ all three **core points**, a second cluster
- **I** = (1, 2.2): distance to D is 1.2 (within ϵ), but I's **own** neighborhood only contains D and itself, only 2 points, below minPts $\rightarrow$ **not core**, but within ϵ of core point D $\rightarrow$ **border point**, joins D's cluster
- **H** = (5,5): nearest other point is roughly 5.7 away, far beyond $\epsilon \rightarrow$ **noise**



When DBSCAN wins over K-means

- Non-spherical, elongated, or irregularly shaped clusters (K-means assumes roughly round blobs)
- Data with genuine outliers that should be **labeled as noise**, not forced into the nearest cluster
- Situations where the “right” number of clusters is unknown and should be discovered, not guessed
- Trade-off: DBSCAN struggles when clusters have very different densities, since ϵ and minPts are global settings



Beyond DBSCAN: OPTICS and HDBSCAN

- **OPTICS** generalizes DBSCAN by ordering points to reveal a range of density thresholds at once, rather than one fixed ϵ
- **HDBSCAN** builds a hierarchy of DBSCAN-style clusterings and picks the most stable clusters across density levels
- Both address DBSCAN's weakness with clusters of varying density, mentioned as pointers, not covered in depth



Clustering methods, side by side

Aspect	K-means	Hierarchical	DBSCAN
Cluster shape	Spherical	Any (depends on linkage)	Arbitrary
Number of clusters	Must choose K	Choose a cut height	Found automatically
Handles noise/outliers	No	No	Yes, labels them as noise
Sensitive to density variation	Somewhat	Somewhat	More
Typical scale	Large datasets	Small-to-medium (all-pairs distances)	Medium



Time complexity across the three families

Method	Typical time complexity	Notes
K-means	$O(n \cdot K \cdot \text{iterations})$	Scales well; mini-batch variants scale further
Hierarchical (naive)	$O(n^2)$ or worse	All-pairs distance matrix, limits n in practice
DBSCAN	$O(n \log n)$ with a spatial index, $O(n^2)$ without	Depends heavily on implementation

When n is large, K-means and DBSCAN (with an index) are the practical choices; hierarchical clustering shines on smaller, exploratory datasets where the whole tree is valuable.



Choosing among K-means, hierarchical, and DBSCAN

- Know K, expect roughly round clusters, large n → **K-means** (or mini-batch K-means)
- Want to explore the data's structure at multiple granularities, n is modest → **hierarchical clustering**
- Expect irregular shapes, expect real noise/outliers, don't know the number of clusters → **DBSCAN**
- Not exclusive, running more than one and comparing is common practice



Check your understanding: the clustering landscape

- In the DBSCAN worked example, why is I a border point rather than a core point?
- Why did switching from single to complete linkage change where the final merge happened, but not the **first** merge?
- If you had to cluster GPS pings from delivery vehicles (irregular shapes, unknown cluster count, likely outliers from bad GPS fixes), which method would you reach for first?



Evaluating unsupervised models

No ground truth, no free lunch

Why evaluation is hard here

- Supervised learning: compare predictions to known labels, the labels **are** the answer key
- Unsupervised learning: often there is no ground truth to compare against at all
- We fall back on **internal** structure (how good is this clustering, on its own terms?), or on labels that happen to exist but were withheld from training



Internal metrics: the silhouette score, formally

- Already introduced during K-means: $\text{silhouette}(i) = (b(i) - a(i)) / \max(a(i), b(i))$
- Works for **any** clustering method, not just K-means, a general-purpose internal metric
- Average over all points to compare different clusterings, or different K, on the same data
- Downside: still assumes a distance-based notion of “cohesive cluster,” which fits K-means-style clusters better than, say, DBSCAN’s irregular shapes



Internal metrics: the Davies-Bouldin index

- For each cluster, computes a ratio of within-cluster scatter to between-cluster separation, then averages the worst-case ratio across cluster pairs
- Lower Davies-Bouldin values indicate better-separated, more compact clusters
- Like silhouette, does not require ground-truth labels, but the two metrics can disagree, since they weight compactness vs. separation differently



Internal metrics: the inertia trap

- Inertia (used inside K-means's own objective) always decreases as K increases, even all the way to $K = n$, where inertia is exactly 0
- Comparing inertia **alone** across different K values will always “recommend” the largest K , which is meaningless
- Only useful when combined with a penalty for K (as in the elbow method) or compared against a fixed K across different initializations



External metrics: the Rand index

Used only when labels happen to be available for validation, never for training:

- Compare every pair of points: do the discovered clusters agree with the true labels on whether this pair is together or apart?
- Rand index = (agreeing pairs) / (all pairs)
- Ranges 0 to 1, but even a **random** clustering scores above 0 by chance, this is a known weakness



External metrics: the adjusted Rand index

- Corrects the raw Rand index for the agreement expected purely by chance
- Adjusted Rand index = 0 for a random clustering (in expectation), 1 for a perfect match, and can go negative for worse-than-chance agreement
- The “adjustment” is exactly why this is preferred over the raw Rand index in practice



External metrics: normalized mutual information

- Measures how much information the discovered cluster assignment shares with the true labels, using entropy
- Normalized to range 0 (no shared information) to 1 (perfect agreement)
- Complements the adjusted Rand index, the two occasionally disagree on which of two clusterings is “better,” which is itself informative

Stability-based validation: bootstrap resampling

- Idea: a good clustering should be **stable**, resample the data (with replacement) and re-cluster; a robust structure should reappear
- Run the clustering algorithm on many bootstrap resamples, measure how consistently the same groups emerge
- High instability across resamples is a warning sign that the “structure” may be an artifact of this particular sample



Practical guidance with zero ground truth

- Sanity-check with domain knowledge: do the discovered groups make sense to someone who knows the data?
- Visualize: project to 2D (via PCA or otherwise) and look
- Compare multiple K values and multiple algorithms; consistent structure across methods is more trustworthy than a result from just one
- Report internal metrics, but do not treat them as an objective substitute for judgment



Putting it together: an evaluation checklist

1. Do the clusters make sense on inspection/visualization?
2. Does an internal metric (silhouette, Davies-Bouldin) support the chosen K or the chosen algorithm over reasonable alternatives?
3. If any labels exist for validation, do external metrics (adjusted Rand index, normalized mutual information) confirm agreement?
4. Is the clustering stable under resampling or reasonable perturbations of the data?



Check your understanding: evaluation

- Why is a rising inertia curve not, by itself, evidence that K is too large?
- What does the adjusted Rand index correct for that the plain Rand index does not?
- You cluster customer purchase histories with no labels at all, walk through how you would still gain confidence in the result



Unsupervised learning in practice

Anomaly detection · Pipelines

Anomaly detection via density and distance

- Points far from every cluster center, or in very low-density regions, are natural anomaly candidates
- DBSCAN's “noise” label is, in effect, a built-in anomaly detector
- Alternative: **isolation forest**, which isolates points using random partitioning, anomalies are isolated in **fewer** splits than normal points (mentioned as a pointer, not covered in depth)
- Anomaly detection is unsupervised almost by necessity: anomalies are rare and their exact form is often unknown in advance



Anomaly detection: a worked example

Reusing the DBSCAN worked example: A, B, C, D form one dense cluster; E, F, G a second; I is a border point; H = (5,5) is flagged as noise.

- H's isolation **is** the anomaly signal, no labels were needed to notice it doesn't belong
- In a fraud-detection setting, H might represent a transaction unlike any typical customer behavior
- The same density reasoning that clusters “normal” behavior also surfaces what does not fit it



Embeddings: unsupervised representations everywhere

- An **embedding** is a learned, lower-dimensional representation of an item (a word, an image, a user)
- Many embeddings are trained with unsupervised or self-supervised objectives (predict a neighboring word, reconstruct a masked patch)
- Once trained, embeddings are frequently **clustered** or **PCA-reduced** for visualization or downstream use
- Preview: the language models lecture builds on exactly this idea, at a much larger scale

A typical pipeline: reduce, then cluster

- Raw features are often noisy, correlated, and high-dimensional
- Common pattern: PCA (or another reducer) first, to strip redundancy and noise, then cluster in the reduced space
- Clustering distances are more meaningful in fewer, less noisy dimensions, this directly addresses the curse of dimensionality from earlier
- The reduced clustering can itself become a feature for a later supervised model



Case study: customer segmentation

- Features: recency, frequency, and monetary value of purchases (an “RFM” feature set)
- Standardize the three features, then run K-means with a handful of candidate K values
- Use the elbow method and silhouette score to pick K, then interpret each cluster’s centroid (e.g. “frequent, recent, high spend” vs. “infrequent, at risk of churn”)
- The output feeds directly into a targeted marketing or retention strategy



Case study: genomics

- Features: expression levels across many genes, per cell or per sample, very high dimensional, often more features than samples
- PCA (or a nonlinear reducer) first, both to fight the curse of dimensionality and to visualize
- Clustering on the reduced representation proposes candidate cell types or sample subgroups
- Follow-up work (often supervised, using known marker genes) validates whether a cluster corresponds to a real biological category



Case study: network intrusion and fraud detection

- Traffic or transaction features are monitored continuously; “normal” behavior vastly outnumbers “attack” or “fraud” behavior
- Cluster or density-model the normal traffic; flag points that do not fit any dense region (DBSCAN’s noise label, or a distance threshold from the nearest cluster)
- Crucial caveat: attackers adapt, so a clustering trained once will drift out of date, this is an ongoing monitoring problem, not a one-time fit



Common pitfalls, recap

- Forgetting to standardize before PCA or K-means, letting one large-scale feature dominate
- Trusting inertia alone to choose K
- Treating a discovered cluster as a “true” category without checking it against domain knowledge
- Applying K-means to non-spherical clusters and being surprised by the result
- Skipping the “does this make sense” sanity check because a metric looked good



Ethical considerations: clustering people

- Clustering customers, patients, or citizens can create de facto categories with real consequences (pricing, eligibility, policing)
- Unsupervised methods can encode and amplify biases already present in the data, without any label ever saying “protected group”
- A cluster is a **statistical** grouping, not a natural or objective category, treat any downstream use of people-clusters with the same scrutiny as a supervised model making decisions about people
- Revisit lecture 1’s fairness and accountability themes here, they apply just as much to unsupervised pipelines



When **not** to use unsupervised learning

- When labels are actually available and cheap, supervised learning will almost always do better with a clear objective
- When the “structure” you are hoping to find has no plausible basis in the data (garbage in, confident-looking garbage out)
- When the stakes are high and a discovered cluster would need to be defensible to a regulator or the people it affects, treat clustering output as a hypothesis to validate, not a verdict



Check your understanding: applications

- Why does an anomaly detector often not need any labeled examples of the anomaly it is trying to catch?
- What is the risk of skipping the “reduce, then cluster” pipeline and clustering directly on hundreds of raw, correlated features?
- Name one ethical risk specific to clustering people, and one mitigation





Wrap-up

Structure, evaluated, connected forward

Unsupervised learning as a building block

- PCA before clustering: reduce noise and dimensionality first
- Embeddings (from later in the course) are themselves an unsupervised-style representation, reused everywhere downstream
- Clustering and anomaly detection are often the first pass before a more targeted, supervised model

Structure discovered here often becomes **input** to the next model, not the final answer.



Today's toolkit, in one table

Tool	Question it answers	Watch out for
PCA	Which few axes capture most of the variance?	Linear only; standardize first
K-means	Which K roughly spherical groups fit best?	Choosing K; sensitive to initialization
Hierarchical clustering	What does the full nested structure look like?	$O(n^2)$ cost; linkage choice matters
DBSCAN	Which dense regions form clusters, and what is noise?	Struggles with varying density
Silhouette / Davies-Bouldin	How good is this clustering, internally?	Not a substitute for domain judgment



Looking ahead

- **Assignment 2** (Unsupervised learning) due October 1 at 3:00 pm
- Next week: **Search**, BFS, DFS, IDS, Dijkstra, A*, and their complexity
- A shift from “finding structure in data we’re given” to “finding a path through structure we define ourselves”, the course’s second thread begins





THANK YOU

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030